

DAY 24: GRADIENT DESCENT OPTIMIZATION

Algorithmic Implementation, Uniqueness Theory, and Framework Verifications

This technical notebook chronicles the quantitative validation workflows conducted during Day 24 of the predictive modeling curriculum. By isolating mathematical architectures under varying structural constraints—specifically transitioning from single-observation instances to multi-variant historic profiles—we evaluate parameters via pure gradient optimization loops alongside certified computational frameworks.

CASE 1: SINGLE-DAY DATA OPTIMIZATION (SINGLE OBSERVATION PROFILING)

Under Case 1, optimization frameworks evaluate a localized constraint boundary comprising an individual record matrix: $X = 2$ mapping precisely to an observed outcome value $Y = 4$. This foundational environment introduces structural optimization logic and mathematical convergence vectors.

1. Manual Algorithmic Loop Implementation (Python)

The core optimization structure leverages sequential differential evaluations to align arbitrary parameter weight metrics towards the target coordinates. Standard localized space features tracking routines and convergence checkpoints.

```

# =====
# CASE 1: SINGLE-DAY DATA GRADIENT DESCENT OPTIMIZATION
# =====

# 1. Target Data Point Configuration
X = 2
actual_y = 4

# 2. Arbitrary Parametric Guesses
m = 1.0
c = 0.5

# 3. Learning Rate Hyperparameter & Iteration Monitor
learning_rate = 0.1
loop_count = 0

print("Initiating Localization Tracking Cycle...")
print(f"Initial State Estimates -> Parameter m: {m:.4f}, Intercept c: {c:.4f}")
print("-" * 65)

while True:
    loop_count += 1

    # Step 1: Computational Model Hypothesis Execution ( $Y = mX + c$ )
    pred_y = (m * X) + c

    # Step 2: Mean Squared Error Cost Assessment
    error = (actual_y - pred_y) ** 2

    # Step 3: Gradient Evaluators (Partial Calculus Derivatives)
    gradient_m = -2 * X * (actual_y - pred_y)
    gradient_c = -2 * (actual_y - pred_y)

    # Print localized sequence trajectory
    print(f"Iteration {loop_count}:")
    print(f"    Predicted Y = {pred_y:.4f} | Instantaneous Loss = {error:.4f}")
    print(f"    Gradient_m = {gradient_m:.4f} | Gradient_c = {gradient_c:.4f}")

    # Convergence Threshold Strategy: Termination upon flat gradient limit
    if abs(gradient_m) < 0.0001 and abs(gradient_c) < 0.0001:
        print("-" * 65)
        print("Convergence Reached! Gradients stabilized near absolute baseline zero.")
        break

    # Step 4: Simultaneous Hyperplane Updates
    m = m - (learning_rate * gradient_m)
    c = c - (learning_rate * gradient_c)

    print(f"    Adjusted Weight Updates -> m = {m:.4f}, c = {c:.4f}")

print("-" * 65)

```

```
print("ALGORITHMIC STABLE OUTPUTS:")
print(f"Total Operational Iterations: {loop_count}")
print(f"Optimized Slope Parameter (m): {m:.2f}")
print(f"Optimized Intercept Metric (c): {c:.2f}")
print(f"Resulting Hypothesis Equation: Y = {m:.2f} * X + {c:.2f}")
```

2. Industrial Library Framework Execution (Scikit-Learn)

To cross-evaluate execution behavior, the exact data instance is fed directly into a localized ordinary least squares framework module.

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Data configuration mapped into required matrix dimensions
X_matrix = np.array([[2]])
y_vector = np.array([4])

# Initialize and train linear estimator model
framework_estimator = LinearRegression().fit(X_matrix, y_vector)

# Output parsed modeling coefficients
print("--- SCYTHE FRAMEWORK EVALUATIONS ---")
print(f"Framework Scaled Coefficient (m): {framework_estimator.coef_[0]:.4f}")
print(f"Framework Scaled Intercept (c): {framework_estimator.intercept_: .4f}")
```

Mathematical Analysis of Indeterminate Spatial Boundaries:

Because a singular coordinate point establishes an isolated target, an infinite series of coordinate hyperplanes can intersect it with absolute zero-loss precision. The linear algebra proof shows that for $4 = 2m + c$, any coordinate pair matching this constraint profile evaluates as valid. The programmatic loop converges at the nearest localized valley relative to its initial initialization path, while the Scikit-Learn framework utilizes an automated algebraic calculation shortcut to output a flat default profile ($m = 0.0, c = 4.0$).

CASE 2: MULTI-DAY DATA OPTIMIZATION (HISTORICAL BATCH PROFILING)

Case 2 expands the geometric requirements by introducing multiple discrete coordinates: **Day 1** ($X=2, Y=4$) and **Day 2** ($X=4, Y=12$). The multi-observation batch landscape requires global matrix adjustments to maintain system balance across all records.

1. Manual Algorithmic Loop Implementation (Python)

To avoid numerical variance spikes and structural exceptions over expanded batch profiles, the updating step index is locked securely at `0.01`, ensuring systematic gradient descents across data environments.

```

# =====
# CASE 2: MULTI-DAY DATA GRADIENT DESCENT OPTIMIZATION
# =====

# 1. Batch Historic Matrix Arrays
X_data = [2, 4]
actual_y_data = [4, 12]

# 2. Baseline Model Parameter Initializations
m = 2.2
c = 0.5

# 3. Guarded Learning Index Coefficient & Iteration Monitor
learning_rate = 0.01
loop_count = 0

print("Initiating Batch-Level Historical Tracking Loop...")
print(f"Initial Balance Metrics -> m = {m:.4f}, c = {c:.4f}")
print("-" * 65)

while True:
    loop_count += 1

    total_error = 0
    sum_gradient_m = 0
    sum_gradient_c = 0

    # Scan across active batch instances sequentially
    for i in range(len(X_data)):
        X = X_data[i]
        actual_y = actual_y_data[i]

        # Predictive execution sequence
        pred_y = (m * X) + c

        # Loss compilation (Mean Square Calculation Step)
        instance_loss = (actual_y - pred_y) ** 2
        total_error += instance_loss

        # Partial Calculus Derivatives compilation
        grad_m = -2 * X * (actual_y - pred_y)
        grad_c = -2 * (actual_y - pred_y)

        sum_gradient_m += grad_m
        sum_gradient_c += grad_c

    # Multi-Data Structural Reality: Computation of Global System Averages
    mse_cost = total_error / len(X_data)
    avg_gradient_m = sum_gradient_m / len(X_data)
    avg_gradient_c = sum_gradient_c / len(X_data)

    # Comprehensive systemic logging output

```

```

print(f"Iteration {loop_count}:")
print(f"    Global Loss Profile (MSE) = {mse_cost:.4f}")
print(f"    Avg Gradient_m = {avg_gradient_m:.4f} | Avg Gradient_c = {avg_gradient_c:.4f}")

# Global Stopping Strategy: Combined convergence checkpoint tracking
if abs(avg_gradient_m) < 0.0001 and abs(avg_gradient_c) < 0.0001:
    print("-" * 65)
    print("Systemic Global Balance Achieved! System halt requested.")
    break

# Simultaneous Global Hyperplane Updates
m = m - (learning_rate * avg_gradient_m)
c = c - (learning_rate * avg_gradient_c)

print(f"    Adjusted Weight Updates -> m = {m:.4f}, c = {c:.4f}")
")

print("-" * 65)
print("FINAL GLOBAL BATCH RECOVERY COMPILATION:")
print(f"Total Operational Iterations: {loop_count}")
print(f"Global Optimized Slope Parameter (m): {m:.2f}")
print(f"Global Optimized Intercept Metric (c): {c:.2f}")
print(f"Unified Global System Equation: Y = {m:.2f} * X + ({c:.2f})")

```

2. Industrial Library Framework Execution (Scikit-Learn)

```

from sklearn.linear_model import LinearRegression

# Standard multi-dimensional array mapping for batch frames
X_batch = [[2], [4]]
y_batch = [4, 12]

# Framework initialization and training pass execution
batch_estimator = LinearRegression().fit(X_batch, y_batch)

print("--- SKLEARN BATCH CORE VERIFICATION ---")
print(f"Verified Global Slope (m): {batch_estimator.coef_[0]:.2f}")
print(f"Verified Global Intercept (c): {batch_estimator.intercept_: .2f}")
print(f"Verified Model Equation: Y = {batch_estimator.coef_[0]:.2f} * X + {batch_estimator.i

```

Geometric Verification and Mathematical Uniqueness Proof:

Unlike individual tracking records, multi-point coordinate systems limit mathematical options to a single operational line. Euclidian geometry states that across two separate points, exactly one distinct unique line can achieve global balance. Because only one global minimum mathematical path exists, the manual gradient loop optimization logic and Scikit-Learn's core solvers converge perfectly on identical parameters: **$m = 4.00$, $c = -4.00$** .